

Mount Realty

Kleegr Project, Building & Unit Inventory System

Rewritten Master Project Specification and Implementation Plan

Prepared for	Mount Realty
Primary platform	Kleegr CRM, backed by the current HighLevel-compatible account and custom objects
Core records	Projects, Buildings, Units, Contacts and Opportunities
Primary deliverables	Import Center, relationship-safe data loading, inventory dashboard, opportunity-to-unit synchronization and reporting
Document status	Updated replacement for the original project setup PDF
Version date	July 10, 2026

Purpose of this document. This is not a copy of the original PDF. It is a newly written, consolidated specification that preserves the original business goal while incorporating the discoveries, corrected import rules and portal decisions made during implementation planning.

Contents

1	Executive Summary	2
2	Product Vision	2
2.1	Business Outcomes	2
3	Guiding Principles	3
4	Scope	3
4.1	Included in the Master Scope	3
4.2	Not Included in the First Import Release	3
5	Confirmed CRM Foundation	3
6	Canonical Data Model	4
6.1	Hierarchy	4
6.2	Required Relationship Rules	4
6.3	Source of Truth	4
7	Stable Identity Strategy	4
7.1	Required Schema Enhancement	5
7.2	Matching Priority	5
8	Unit Inventory Lifecycle	5
8.1	Consistency Rules	5
9	Inventory Count Rules	6
9.1	Count Fields	6
10	Import Center	6
10.1	Portal Flow	6
10.2	Supported Batch Types	7
10.2.1	Project + Buildings + Units	7
10.2.2	Building + Units	7
10.2.3	Units Only	7
10.3	Required Upload Columns	7
10.4	Allowed Values	8
10.5	Validation Rules	8
10.6	Preview Requirements	9
11	Import Execution	9
11.1	Order of Operations	9
11.2	Idempotency	10
11.3	Partial Failure Policy	10
11.4	Final Import Report	10
12	Inventory Portal and Dashboard	10
12.1	Dashboard	10
12.2	Inventory Browser	10
12.3	Manual Corrections	11
13	Opportunity-to-Unit Synchronization	11
13.1	Association Requirement	11
13.2	Stage Mapping	11
13.3	Double-Reservation Protection	11
13.4	Reversal Rules	11
14	Backend Architecture	12

14.1 Recommended Components	12
14.2 Frontend Responsibilities	12
14.3 Backend Responsibilities	12
15 Security and Data Protection	12
16 Reliability and Observability	12
17 AI Studio Role and Limitation	13
18 Implementation Roadmap	13
18.1 Phase 0: Discovery and Contract Verification	13
18.2 Phase 1: Backend Foundation	13
18.3 Phase 2: Parser, Template and Validation	13
18.4 Phase 3: CRM Upsert Engine	13
18.5 Phase 4: Associations and Rollups	14
18.6 Phase 5: Inventory Dashboard	14
18.7 Phase 6: Opportunity Synchronization	14
18.8 Phase 7: QA and Production Hardening	14
19 Acceptance Criteria	14
20 QA Test Scenarios	15
21 Developer Completion Report	15
22 Open Decisions and Risks	16
23 Immediate First Assignment	16
24 Appendix A: Confirmed V1 Field Mapping	16
24.1 Project	17
24.2 Building	17
24.3 Unit	17
25 Appendix B: Definition of Done	17

1. Executive Summary

Mount Realty needs one reliable inventory system for real estate Projects, Buildings and Units inside Kleeqr. The CRM already contains three custom objects for this hierarchy. The finished system must allow administrators to load inventory in bulk, connect every record correctly, keep unit availability accurate, connect sales opportunities to units, calculate inventory totals automatically and provide a clear operational portal.

The solution is not simply an Excel uploader. It is a controlled inventory data system with five responsibilities:

1. Receive structured inventory files and validate them before any CRM write occurs.
2. Create or update Projects, Buildings and Units without duplication.
3. Build the correct parent-child relationships between Project, Building and Unit records.
4. Treat Unit status as the operational source of truth and roll totals upward to Buildings and Projects.
5. Synchronize an Opportunity's sales stage with the associated Unit without double-counting or silently corrupting inventory.

Critical implementation reality. A visual AI Studio prototype can design the screens, but a frontend-only application cannot safely hold the CRM token or perform reliable production imports. The real system requires a server-side backend for file parsing, validation, API calls, retries, audit logging and import execution.

The first usable release should prioritize the Import Center and trustworthy CRM writes. The inventory dashboard and opportunity synchronization should be built only after object creation, matching and associations have been proven against the live account.

2. Product Vision

The product should feel like a simple internal Kleeqr application used by office staff every day. A user should be able to open the portal, upload one inventory batch, understand exactly what will happen, fix errors before import and receive a complete report afterward.

The system should eliminate these common operational problems:

- Duplicate Projects, Buildings or Units created by repeat uploads.
- Units attached to the wrong Building or Project.
- Inventory counts manually typed and becoming inconsistent.
- Sales opportunities reserving or selling the same Unit more than once.
- Staff changing a pipeline stage without the Unit record reflecting the change.
- Failed imports that leave the user unsure which records were created.
- API failures hidden behind a generic success message.

2.1 Business Outcomes

The completed system should provide:

- One searchable source of truth for all active real estate inventory.
- A repeatable upload process for a full Project, a single Building or standalone Units.
- Accurate counts for total, available, reserved, under-contract and sold inventory.
- Clear visibility from Project to Building to Unit and from Opportunity to Unit.
- Safe re-imports that update existing records instead of duplicating them.
- An auditable history of every import and every automated inventory change.

3. Guiding Principles

1. **Validate before writing.** Uploading a file must never immediately write to the CRM.
2. **Units are the count source.** Project and Building totals are calculated from Unit records, not trusted from spreadsheets.
3. **Relationships use stable IDs.** Names are labels, not dependable relationship keys.
4. **Re-imports must be idempotent.** Processing the same file twice must not duplicate records.
5. **One active Unit stage.** Even if the CRM field technically supports multiple selections, the business system permits only one active inventory stage.
6. **No hidden partial success.** Every failed or skipped record must appear in the import report.
7. **Server-side secrets only.** Tokens and credentials never belong in browser code, screenshots, downloadable reports or logs.
8. **Live CRM data must be labeled honestly.** Simulated or sample data must never be presented as a completed integration.

4. Scope

4.1 Included in the Master Scope

- CRM discovery and API contract verification.
- Project, Building and Unit custom-object integration.
- Excel and CSV upload portal.
- Online template viewer and static template download link.
- Validation, preview, matching, create/update and import reports.
- Project-to-Building and Building-to-Unit associations.
- Opportunity-to-Unit association.
- Seller Contact-to-Project association where supported and approved.
- Unit inventory status rules and count rollups.
- Project, Building and Unit search and detail views.
- Opportunity-driven Unit status automation.
- Import history, API diagnostics, retries and audit trail.
- Desktop-first administration with usable mobile review screens.

4.2 Not Included in the First Import Release

- Rebuilding the entire Kleeqr CRM.
- Replacing Contacts, Conversations, Calendars or the standard opportunity pipeline.
- Public customer-facing inventory pages.
- Payments, contracts, invoicing or document generation.
- Automatic data cleanup of unrelated historical CRM records.
- Unreviewed deletion of existing Projects, Buildings, Units or Opportunities.

5. Confirmed CRM Foundation

The discovery report confirmed the following custom objects and identifiers. These values should be configuration, not scattered hard-coded strings.

Object	Object key	Object ID
Project	<code>custom_objects.projects</code>	6a3d6638a203481f18d46483
Building	<code>custom_objects.buildings</code>	6a47df17cac2ac453506991a
Unit	<code>custom_objects.units</code>	6a47dfc4416f7e873476b759

The discovered CRM location is `UpjC8IK37wMzeb1pc9D0`. The Mount Realty pipeline was also readable during discovery. Pipeline and stage identifiers should be loaded dynamically from the CRM so the system does not break when an administrator renames or reorders a stage.

Association warning. The first discovery attempt did not reliably return the object associations. Therefore, the relationship endpoint, association keys and direction must be confirmed with read-only tests and a small controlled write test before bulk import is enabled.

6. Canonical Data Model

6.1 Hierarchy

The intended hierarchy is:

Seller Contact → **Project** → **Building** → **Unit**

An Opportunity may be associated directly with one Unit:

Opportunity → **Unit**

The Project and Building levels are optional for standalone Unit imports, but when a Project exists it must contain at least one Building, and every Building must contain at least one Unit.

6.2 Required Relationship Rules

- One Project may contain many Buildings.
- One Building belongs to no more than one Project in the operational model.
- One Building may contain many Units.
- One Unit belongs to no more than one Building in the operational model.
- One active sales Opportunity should reserve no more than one Unit unless a future multi-unit deal type is explicitly designed.
- A Unit must not be actively reserved by more than one open Opportunity.
- A Seller Contact may be connected to one or more Projects if the CRM association model permits it.

6.3 Source of Truth

The Unit record is the source of truth for inventory state. Project and Building counts are derived summaries. The spreadsheet is a source of incoming data, not the permanent source of truth after import.

The field `inventory_deducted` is a guard and audit indicator. It must not replace recalculation from Unit stage. This prevents a stale Yes/No value from corrupting totals.

7. Stable Identity Strategy

The upload needs four IDs with different jobs. A single universal ID is not enough because it would collapse multiple Buildings and Units into one relationship group.

ID	Uniqueness	Purpose
Import Row ID	Unique per row	Identifies the source line for validation, errors and reports.
Project Import ID	Shared by Project rows	Groups every Building and Unit belonging to the same Project.
Building Import ID	Shared by Building rows	Groups every Unit belonging to the same Building.
Unit Import ID	Unique per Unit	Identifies one Unit for create/update and future re-imports.

7.1 Required Schema Enhancement

Reliable re-imports require permanent external IDs. Before production launch, add or confirm fields equivalent to:

- Project External Import ID on Project.
- Building External Import ID on Building.
- Unit External Import ID on Unit.

If these fields cannot be added, the backend must maintain a durable mapping table between external import IDs and CRM record IDs. Session-only mapping is not sufficient because a later file must be able to update the same record safely.

7.2 Matching Priority

1. Match by permanent External Import ID.
2. If no external ID exists, match Project by Project Code and Building by Building Code.
3. For Units, use Unit External Import ID. As a temporary fallback only, match by Building plus Unit Number.
4. Names alone are not safe matching keys.
5. Any ambiguous match becomes a blocking error and requires human resolution.

8. Unit Inventory Lifecycle

The portal and workflows must normalize Unit status into the following model.

Business state	Availability field	Stage field	Other behavior
Available	Available	Blank	Inventory Deducted = No; Locked Date blank unless preserved for history.
Reserved / Locked	Not Available	Reserved/Locked	Inventory Deducted = Yes; set Locked Date when first reserved.
Under Contract	Not Available	Under Contract	Inventory Deducted = Yes.
Closed / Sold	Not Available	Closed/Sold	Inventory Deducted = Yes.

8.1 Consistency Rules

- Available Units must have a blank Stage.
- A Unit with Reserved/Locked, Under Contract or Closed/Sold must be Not Available.
- Multiple active stage values are rejected.

- A sold Unit cannot be returned to Available automatically. Reversal requires an authorized manual action with a reason.
- Moving backward from Under Contract to Reserved may be automated only when the opportunity mapping is unambiguous.
- Cancelling a reservation may release the Unit, but only if no other active Opportunity is associated with it.

9. Inventory Count Rules

Counts must be recalculated from Unit records after every import and every relevant Opportunity change.

For each Building:

- Total Units = count of all associated Units.
- Available Units = Availability is Available and Stage is blank.
- Reserved / Locked Units = Stage is Reserved/Locked.
- Under Contract Units = Stage is Under Contract.
- Sold Units = Stage is Closed/Sold.

For each Project, apply the same rules across all Units in all associated Buildings.

9.1 Count Fields

Project rollups write to:

- `custom_objects.projects.total_units`
- `custom_objects.projects.available_units`
- `custom_objects.projects.reserved__locked_units`
- `custom_objects.projects.under_contract_units`
- `custom_objects.projects.sold_units`
- `custom_objects.projects.recalc_requested`

Building rollups write to:

- `custom_objects.buildings.total_units`
- `custom_objects.buildings.available_units`
- `custom_objects.buildings.reserved__locked_units`
- `custom_objects.buildings.under_contract_units`
- `custom_objects.buildings.sold_units`
- `custom_objects.buildings.recalc_requested`

If Total Units does not equal the sum of the four status buckets, the dashboard must show an Unclassified count and identify the inconsistent Units. It must not hide the discrepancy.

10. Import Center

10.1 Portal Flow

The Import Center should contain these steps:

1. Template and instructions.
2. Upload file.

3. Parse and validate.
4. Review Project, Building and Unit previews.
5. Resolve blocking errors.
6. Confirm import.
7. Execute server-side job.
8. Display and download the final report.
9. Store import history for later review and retry.

The preferred format is XLSX. XLS and CSV may be accepted if parsing is reliable. Only the sheet named **Inventory Import** should be processed from an Excel workbook.

10.2 Supported Batch Types

Project + Buildings + Units

Exactly one Project Import ID is permitted in the file. The Project must contain one or more Buildings, and every Building must contain one or more Units.

Building + Units

Exactly one Building Import ID is permitted in the file. No Project is required. The Building must contain one or more Units.

Units Only

One or more standalone Units may be uploaded. Project Import ID and Building Import ID may be blank.

Batch rule. A Project cannot be uploaded without Buildings and Units. A Building cannot be uploaded without Units. Units may be uploaded without either parent.

10.3 Required Upload Columns

The canonical upload contains 30 columns. Each row represents one Unit.

No.	Column	Rule	Purpose / mapping
1	Import Row ID	Required	Unique source-row identifier.
2	Import Mode	Required	Project + Buildings + Units; Building + Units; or Units Only.
3	Project Import ID	Conditional	Required in Project mode; shared by all rows in the Project.
4	Building Import ID	Conditional	Required in Project and Building modes; shared by Units in the Building.
5	Unit Import ID	Required	Unique stable Unit identifier.
6	Project Name	Conditional	Maps to <code>custom_objects.projects.projects</code> .
7	Project Code	Conditional	Maps to <code>custom_objects.projects.project_code</code> .
8	Project Status	Optional	Maps to <code>custom_objects.projects.project_status</code> .
9	Project Property Type	Optional	Maps to <code>custom_objects.projects.property_type</code> .
10	Project Address	Optional	Maps to <code>custom_objects.projects.address</code> .
11	Building Name	Conditional	Maps to <code>custom_objects.buildings.building_name</code> .
12	Building Code	Optional	Maps to <code>custom_objects.buildings.building_code</code> .
13	Building Address	Optional	Maps to <code>custom_objects.buildings.building_address</code> .
14	Building Status	Optional	Maps to <code>custom_objects.buildings.building_status</code> .
15	Unit Name	Required	Maps to <code>custom_objects.units.unit_name</code> .
16	Unit Number	Required	Maps to <code>custom_objects.units.unit_number</code> .

No.	Column	Rule	Purpose / mapping
17	Available / Not Available	Optional	Maps to <code>custom_objects.units.availablenot_available</code> .
18	Stage	Optional	Maps to <code>custom_objects.units.stages</code> .
19	Rooms	Optional	Maps to <code>custom_objects.units.rooms</code> .
20	Bedrooms	Optional	Maps to <code>custom_objects.units.bedrooms</code> .
21	Floor	Optional	Maps to <code>custom_objects.units.floor</code> .
22	Style	Optional	Maps to <code>custom_objects.units.style</code> .
23	Total SF	Optional	Maps to <code>custom_objects.units.total_sf</code> .
24	Move-in Ready	Optional	Maps to <code>custom_objects.units.movein_ready</code> .
25	Asking / Sale Price	Optional	Maps to <code>custom_objects.units.asking__sale_price</code> .
26	Price Per SF	Optional	Maps to <code>custom_objects.units.price_per_sf</code> .
27	Inventory Deducted?	Optional	Maps to <code>custom_objects.units.inventory_deducted</code> .
28	Locked Date	Optional	Maps to <code>custom_objects.units.locked_date</code> .
29	Import Notes	Optional	Stored in import history unless a CRM field is approved.
30	Skip Row?	Optional	Yes means validate and report, but do not import.

10.4 Allowed Values

Project Status

- Active
- Nearly Sold Out
- Sold Out
- Coming Soon
- Inactive
- Data Review Needed

Project Property Type

- Condo
- Rental
- Mixed Use

Building Status

- Active
- Coming Soon
- Partially Available
- Reserved / Locked
- Under Contract

- Sold Out

- Inactive

Unit Availability

- Available
- Not Available

Unit Stage

- Reserved/Locked
- Under Contract
- Closed/Sold

Unit Style

- Flat
- L Flat
- Up & Down
- Walk In
- 3 Story
- Other
- Unknown

10.5 Validation Rules

Blocking errors include:

- Missing or changed header names.
- Duplicate Import Row ID.
- Duplicate Unit Import ID.
- More than one Project Import ID in Project mode.

- More than one Building Import ID in Building mode.
- Project rows without a Building or Unit.
- Building rows without a Unit.
- Missing required names, codes or Unit numbers for the selected mode.
- Invalid option values.
- Multiple Unit stages on one row.
- Contradictory availability and stage values.
- Ambiguous CRM match.
- A Unit Import ID already mapped to a different Building.
- A Building Import ID already mapped to a different Project.

Warnings include:

- Missing optional price, square footage, floor, style or address.
- Price Per SF differs materially from Price divided by Total SF.
- A record name changed while the stable external ID stayed the same.
- A Project or Building status conflicts with calculated inventory state.
- An existing CRM record contains values that will be replaced.

10.6 Preview Requirements

Before import, show summary cards for total rows, skipped rows, Projects, Buildings, Units, creates, updates, warnings and blocking errors.

Preview tabs must include Projects, Buildings, Units, Errors and Warnings. Each preview record should show:

- Action: Create, Update, Skip or Error.
- Source Import Row IDs.
- Matching CRM record ID when found.
- Existing values and proposed values for changed fields.
- Parent association that will be created.
- Reason for every warning, skip or error.

11. Import Execution

11.1 Order of Operations

After explicit confirmation, execute in this order:

1. Create an Import Job with file hash, user, timestamp and validation snapshot.
2. Lock the job against duplicate execution.
3. Create or update the Project when applicable.
4. Create or update the Building when applicable.
5. Create or update all Units.
6. Create Project-to-Building associations.
7. Create Building-to-Unit associations.

8. Recalculate Building totals.
9. Recalculate Project totals.
10. Verify the final CRM state by reading records back.
11. Finalize the Import Report.

11.2 Idempotency

The backend must calculate a file hash and use stable external IDs. Re-running the same confirmed job should return the prior result unless the user deliberately creates a new revision. API retries must be safe and must not create duplicate records or duplicate associations.

11.3 Partial Failure Policy

A failed Unit must not make the system report the entire file as successful. The job should continue where safe, then finish as Success, Success With Warnings, Partial Failure or Failed.

The report must identify exactly which records were created before failure. Retrying the failed items must use the same external IDs and update or continue, not duplicate.

11.4 Final Import Report

The report should include:

- Import job ID, filename, file hash, user and timestamps.
- Total rows, skipped rows, warning count and error count.
- Projects created and updated.
- Buildings created and updated.
- Units created and updated.
- Association results.
- Count recalculation results.
- API failures with safe error details and request correlation IDs.
- A downloadable CSV and copyable text summary.

12. Inventory Portal and Dashboard

After the Import Center is stable, build the inventory workspace.

12.1 Dashboard

The dashboard should show:

- Total Projects, Buildings and Units.
- Available, Reserved/Locked, Under Contract and Sold counts.
- Units with inconsistent status.
- Recent imports and failed jobs.
- Projects with recalculation requested or mismatched totals.
- Units reserved without a valid Opportunity association.

12.2 Inventory Browser

Users should be able to:

- Search by Project, Building, Unit name, Unit number, code or address.

- Filter by Project, Building, availability, stage, rooms, bedrooms, floor, style, price and square footage.
- Expand a Project to see Buildings and expand a Building to see Units.
- Open a Unit detail page showing inventory state, price, physical details, parent records and linked Opportunity.
- Open the corresponding CRM record in a new tab.
- Export the current filtered inventory view.

12.3 Manual Corrections

Authorized users may correct a Unit, but every change must show the previous value, new value, actor, timestamp and reason. High-risk changes such as Sold to Available should require a confirmation step and should trigger count recalculation.

13. Opportunity-to-Unit Synchronization

13.1 Association Requirement

An Opportunity must be associated with a Unit before inventory automation can run. The Opportunity should also remain connected to its Contact through the standard CRM model.

13.2 Stage Mapping

The exact pipeline stage mapping must be configured from the live pipeline. Do not assume that the visible stage name alone is permanent. Store pipeline ID and stage ID, while displaying the current stage name to users.

A typical mapping is:

- Reservation / lock stage → Unit Reserved/Locked.
- Contract stage → Unit Under Contract.
- Closed / won stage → Unit Closed/Sold.
- Cancelled / lost stage → controlled release review.

The current Mount Realty pipeline contains general sales-process stages as well as later contract and customer steps. A business owner must approve which exact stage IDs change inventory. The system must not infer this from similar wording.

13.3 Double-Reservation Protection

Before reserving a Unit, the backend must check:

- The Unit is currently Available.
- No other open Opportunity has an active reservation on the Unit.
- The Opportunity-to-Unit association is unique and valid.
- The transition is not a duplicate event already processed.

If any check fails, do not overwrite the Unit. Create an exception record and alert an administrator.

13.4 Reversal Rules

A lost or cancelled Opportunity may release a reserved Unit only when:

- The Unit is not sold.
- No other active Opportunity depends on the reservation.
- The workflow event is newer than the last processed event.

- The user or approved automation provides a release reason.

14. Backend Architecture

14.1 Recommended Components

- Kleeqr-branded React frontend for the portal.
- Server-side API layer using TypeScript and Node.js.
- Secure environment variables for CRM credentials.
- Relational database for import jobs, external-ID mappings, audit logs and retry state.
- Background job worker for large imports and rate-limited CRM writes.
- Object storage or a private temporary upload location for source files.
- Central CRM client with timeouts, retries, rate-limit handling and safe logging.

14.2 Frontend Responsibilities

The browser may upload files, display previews, request confirmation and show reports. It must not contain the CRM token, call privileged write endpoints directly or claim an import succeeded before the backend verifies it.

14.3 Backend Responsibilities

The backend handles file parsing, validation, matching, CRM reads and writes, associations, rollups, job locking, audit logs, retries and final verification.

15. Security and Data Protection

- Store tokens only in server-side secret storage.
- Rotate the temporary full-access token after development.
- Never place a token in frontend JavaScript, local storage, screenshots or downloadable reports.
- Redact authorization headers, phone numbers and sensitive record data from logs.
- Require authenticated users and role-based permission for imports and manual corrections.
- Restrict file type, file size, row count and formula content.
- Treat spreadsheet cells beginning with formula characters as untrusted when exporting CSV.
- Validate every CRM record belongs to the configured location before updating it.
- Keep an audit trail for writes, associations, status changes and rollbacks.
- Separate test and production environments.

16. Reliability and Observability

The system should include:

- API timeouts and retry rules for transient failures.
- Respect for CRM rate limits and Retry-After responses.
- Per-job progress and heartbeat.
- Dead-letter handling for items that fail repeatedly.
- Correlation IDs connecting the portal, backend job and CRM requests.

- Health checks for the application, database, queue and CRM connectivity.
- Metrics for import duration, API errors, duplicate prevention and association failures.
- Safe resume after worker restart.
- A reconciliation command that recalculates every Project and Building from Units.

17. AI Studio Role and Limitation

AI Studio may be used to prototype the portal interface and generate frontend components. It should not be treated as proof of live CRM integration when its environment is frontend-only.

A production-ready result requires one of these outcomes:

1. AI Studio supports secure server functions and the token remains server-side; or
2. The AI Studio frontend calls a separately deployed authenticated backend; or
3. The portal is moved to a full-stack deployment platform.

The earlier simulated import screen should be considered disposable. Build from a clean specification, and label any simulated behavior until the backend is connected.

18. Implementation Roadmap

18.1 Phase 0: Discovery and Contract Verification

- Confirm all object field keys and data types.
- Confirm association endpoints, keys and direction.
- Confirm create, search, update and pagination behavior for all three custom objects.
- Confirm pipeline and stage IDs.
- Confirm Opportunity-to-Unit association capability.
- Create controlled test records and remove them after verification.

Exit gate: A written API contract and successful read/create/update/associate test for one Project, one Building and two Units.

18.2 Phase 1: Backend Foundation

- Secure configuration and authentication.
- CRM client with retries and redacted logs.
- Database tables for Import Job, Import Item, External ID Mapping and Audit Event.
- File upload storage and job queue.

18.3 Phase 2: Parser, Template and Validation

- Parse XLSX, XLS and CSV.
- Implement the 30-column schema.
- Validate batch hierarchy, IDs, values and consistency.
- Build preview screens and error export.

18.4 Phase 3: CRM Upsert Engine

- Add permanent external import ID fields or mapping table.
- Create/update Projects, Buildings and Units.

- Prove safe re-import and duplicate prevention.
- Verify every write by reading the record back.

18.5 Phase 4: Associations and Rollups

- Create Project-to-Building and Building-to-Unit associations.
- Add count recalculation.
- Build reconciliation and mismatch reporting.

18.6 Phase 5: Inventory Dashboard

- Summary dashboard, hierarchy browser, filters and Unit detail.
- Manual correction workflow with audit trail.
- Import history and retry interface.

18.7 Phase 6: Opportunity Synchronization

- Confirm Opportunity-to-Unit association.
- Approve pipeline stage mapping.
- Add reservation, contract, sold and controlled release workflows.
- Add double-reservation protection and exception queue.

18.8 Phase 7: QA and Production Hardening

- Automated unit, integration and end-to-end tests.
- Load test with realistic inventory files.
- Token rotation, backup, rollback and recovery drills.
- Staff acceptance testing and written operating instructions.

19. Acceptance Criteria

The Import Center is accepted only when all of the following are proven:

1. A valid Project file creates one Project, multiple Buildings and all Units with correct associations.
2. A valid Building file creates one Building and its Units without requiring a Project.
3. A valid Units Only file creates standalone Units.
4. A Project without Buildings or Units is blocked.
5. A Building without Units is blocked.
6. A file with two Project Import IDs is blocked in Project mode.
7. Duplicate Import Row IDs and Unit Import IDs are blocked.
8. Re-importing the same file does not create duplicates.
9. Updating a value under the same External Import ID updates the correct CRM record.
10. Ambiguous matches are blocked rather than guessed.
11. Every association is verified after write.
12. Project and Building totals equal the derived Unit counts.
13. A partially failed import identifies every successful and failed item.
14. No token appears in browser source, logs or reports.

15. A user without import permission cannot execute a job.

The Opportunity automation is accepted only when:

1. One configured pipeline stage reserves a Unit exactly once.
2. A second Opportunity cannot reserve the same Unit.
3. Contract and sold transitions update the Unit correctly.
4. Replayed workflow events do not double-change inventory.
5. Cancellation cannot automatically reopen a sold Unit.
6. Every automated change has an audit event and count recalculation.

20. QA Test Scenarios

At minimum, test:

- One Project, one Building, one Unit.
- One Project, multiple Buildings, many Units.
- Building-only batch.
- Units-only batch.
- Duplicate file replay.
- Renamed Project with same external ID.
- Unit moved to a different Building, which should block pending review.
- Invalid option spelling and extra whitespace normalization.
- Missing required fields by mode.
- Contradictory Available plus Reserved stage.
- CRM API timeout halfway through a job.
- Rate-limit response and delayed retry.
- Worker restart during import.
- Association failure after successful record creation.
- Count mismatch and reconciliation.
- Opportunity reservation conflict.
- Cancelled reservation and sold reversal protection.

21. Developer Completion Report

After every development phase, the developer should return:

1. Objective completed.
2. Files and services created or changed.
3. CRM endpoints used.
4. Field keys and association keys confirmed.
5. Screens completed.
6. Tests run and results.
7. Sample data used.

8. Known limitations and blocked items.
9. Security review notes.
10. Deployment status.
11. Exact manual testing steps for Mount Realty.
12. Recommended next phase.

Claims such as “connected,” “working” or “imported” must include evidence from live read-back or test results. A frontend toast is not evidence.

22. Open Decisions and Risks

Decision / risk	Required resolution
Association API not confirmed	Confirm exact endpoints, association keys, direction and permission before bulk writes.
Permanent external IDs	Add CRM fields or approve a durable backend mapping table. Production importer should not launch without one.
Pipeline stage mapping	Business owner must approve exact stage IDs that reserve, contract, sell and release inventory.
Project and Building status automation	Decide which statuses remain manual and which may be derived from Units.
Standalone Units	Decide whether they remain unassociated permanently or enter a review queue for later assignment.
Historical data cleanup	Decide whether existing duplicate or inconsistent records are included in a separate migration project.
Import size	Set a supported row limit and background-job timeout based on real API rate limits.
AI Studio hosting	Confirm secure server capability or deploy a separate backend.

23. Immediate First Assignment

Before additional portal features are built, the developer should complete and document this discovery gate:

1. Read every current pipeline and stage.
2. Read the Project, Building and Unit schemas, field IDs, field keys, types and options.
3. Confirm existing data volume and identify duplicate or incomplete records.
4. Confirm association definitions and API behavior.
5. Confirm create, update, search, pagination and rate-limit behavior.
6. Confirm available workflow triggers and webhooks for Opportunity stage changes.
7. Confirm whether AI Studio can securely call server-side functions.
8. Produce a final integration matrix showing Confirmed, Limited, Blocked or Requires Workaround.

Only after this gate passes should the developer enable confirmed CRM writes in the Import Center.

24. Appendix A: Confirmed V1 Field Mapping

24.1 Project

Project Name	<code>custom_objects.projects.projects</code>
Project Code	<code>custom_objects.projects.project_code</code>
Project Status	<code>custom_objects.projects.project_status</code>
Property Type	<code>custom_objects.projects.property_type</code>
Address	<code>custom_objects.projects.address</code>
Total Units	<code>custom_objects.projects.total_units</code>
Available Units	<code>custom_objects.projects.available_units</code>
Reserved / Locked Units	<code>custom_objects.projects.reserved__locked_units</code>
Under Contract Units	<code>custom_objects.projects.under_contract_units</code>
Sold Units	<code>custom_objects.projects.sold_units</code>
Recalc Requested	<code>custom_objects.projects.recalc_requested</code>

24.2 Building

Building Name	<code>custom_objects.buildings.building_name</code>
Building Code	<code>custom_objects.buildings.building_code</code>
Building Address	<code>custom_objects.buildings.building_address</code>
Building Status	<code>custom_objects.buildings.building_status</code>
Total Units	<code>custom_objects.buildings.total_units</code>
Available Units	<code>custom_objects.buildings.available_units</code>
Reserved / Locked Units	<code>custom_objects.buildings.reserved__locked_units</code>
Under Contract Units	<code>custom_objects.buildings.under_contract_units</code>
Sold Units	<code>custom_objects.buildings.sold_units</code>
Recalc Requested	<code>custom_objects.buildings.recalc_requested</code>

24.3 Unit

Unit Name	<code>custom_objects.units.unit_name</code>
Unit Number	<code>custom_objects.units.unit_number</code>
Availability	<code>custom_objects.units.availablenot_available</code>
Stage	<code>custom_objects.units.stages</code>
Rooms	<code>custom_objects.units.rooms</code>
Bedrooms	<code>custom_objects.units.bedrooms</code>
Floor	<code>custom_objects.units.floor</code>
Style	<code>custom_objects.units.style</code>
Total SF	<code>custom_objects.units.total_sf</code>
Move-in Ready	<code>custom_objects.units.movein_ready</code>
Asking / Sale Price	<code>custom_objects.units.asking__sale_price</code>
Price Per SF	<code>custom_objects.units.price_per_sf</code>
Inventory Deducted	<code>custom_objects.units.inventory_deducted</code>
Locked Date	<code>custom_objects.units.locked_date</code>

25. Appendix B: Definition of Done

The project is done when Mount Realty staff can import inventory, trust the relationships and totals, see live inventory clearly and move an Opportunity through approved stages without creating a duplicate reservation or requiring manual count adjustments.

A polished interface without secure backend writes is not done. Records created without verified associations are not done. Counts that depend on manual spreadsheet totals are not done. A workflow that can reserve the same Unit twice is not done.

Final standard. The system should be simple for staff, strict with data, honest about failures and safe to run repeatedly.